

Auto-Dock It

Agentic LLM tool that turns any public GitHub repo into a working, validated Docker setup.

Author: Melvin Joshua · Report date: 2026-05-30

Live preview	https://auto-dock-it.streamlit.app
Source	https://github.com/MelvinJoshua1375/auto-dock-it
License	MIT
Continuous integration	passing on Python 3.10, 3.11, 3.12, 3.13
Tests	57 unit tests, ruff lint clean, Bandit security scan clean

Executive summary

Most open-source GitHub projects are hard to run on a fresh machine. READMEs go stale, dependency lists drift, and the setup is rarely documented end to end. Auto-Dock It clones any public repository, figures out its language and framework, generates a Dockerfile (plus a `docker-compose.yml` when the project needs sidecar services like Postgres or Redis), builds it, and confirms the resulting container actually responds on its port. When something fails, the LLM reads the build or container logs and proposes a fix. The loop is bounded, audited to disk, and demonstrably effective on six different test cases.

The gap this fills

Three classes of tooling already exist in this space, each with a known weakness:

Class	Examples	Weakness
Rule-based containerizers	Nixpacks, Cloud Native Buildpacks, repo2docker	Break on unusual stacks or non-standard project layouts
One-shot LLM prompts	"Write me a Dockerfile" demos, blog snippets	No validation; the user finds out it does not build only when they try it
Manual onboarding	Hand-written <code>docker-compose.yml</code> , dev container files	Depends on the maintainer keeping documentation in sync, which rarely happens

Auto-Dock It treats Dockerfile generation as an **agentic loop**. When `docker build` fails, the truncated error log and the current Dockerfile go back to the language model with a request for a patch. The build is retried. If the container builds but the app fails to respond, the container logs are fed back for a second class of repair. Every attempt is saved to disk so the loop is auditable, not a black box.

Approach

Five stages run in order. Each stage writes artifacts so the entire run is reproducible, inspectable, and ready to ship as a PR back to the upstream repo.

1. **Ingest.** Shallow clone the repo with a 200 MB size cap.
2. **Analyze.** Walk the file tree, read manifest files (`requirements.txt` , `package.json` , `pom.xml` , and so on), scan source code for env-var references (`os.environ.get` , `process.env.*` , equivalents in eight languages), and send a structured snapshot to the LLM. The model returns a typed Pydantic `RepoProfile` describing language, framework, port, run command, env vars, and any required external services. Cached by commit SHA so re-runs skip the LLM call.
3. **Generate.** Produce a Dockerfile from the profile. If the profile lists services, also produce `docker-compose.yml` .
4. **Build (self-healing).** Run `docker build` . On failure, send error tail + Dockerfile to the LLM, get a patched Dockerfile, retry. Default budget: four attempts.
5. **Validate.** Run the container or compose stack, poll the app port for any HTTP 2xx or 3xx. On failure, run up to two runtime-repair cycles, feeding container logs back to the LLM.

```
github URL → Ingest → Analyze → Generate → Build (loop) → Validate
              |         |         |         |         |
              clone    profile   Dockerfile  docker build  docker run
              depth 1   from manifest + yaml  & LLM repair  + HTTP poll
                                      & LLM repair

every artifact saved under output/<run_id>/
```

What it builds, demonstrated

Six runs are committed to the repository under `demos/` with the full attempt history, the final Dockerfile, the structured profile, and token usage. The pipeline succeeded on every one.

Demo	Stack	Build attempts	Outcome
flask	Python, Flask, gunicorn	2 (1 repair)	HTTP 200
nodejs	Node, Express	2 (1 repair)	HTTP 200
broken-flask	Flask with a deliberate typo (<code>flsk</code> instead of <code>flask</code> in <code>requirements.txt</code>)	4 (3 repairs)	HTTP 200. The LLM patched the typo with a <code>sed</code> inside the Dockerfile at build time.
flask-redis	Flask plus Redis as a sidecar	1	HTTP 200, auto-generated <code>docker-compose.yml</code> .
env-required-flask	Flask that hard-requires <code>REQUIRED_SECRET</code> at import time, never declared in any manifest	2 (1 repair)	HTTP 200. The source-code env-var scan detected the variable; the LLM added it as <code>ENV</code> in the Dockerfile.
crashing-route-flask	Flask whose route reads a hard-coded <code>/etc/...</code> path that is not in the repo	1	HTTP 200. The LLM read <code>app.py</code> , spotted the path, and added a <code>RUN mkdir -p ... && touch ...</code> to the Dockerfile.

The two final demos were designed to force the runtime-repair safety net. The build-time loop caught both before runtime, which is itself a result worth reporting: the source-code grep plus a well-tuned prompt lets the system fix infrastructure bugs before the container ever runs.

What the loop actually fixes

A representative excerpt from `demos/flask` : attempt 0 fails because

`adduser --system --no-create-home appuser` on `python:3.12-slim` does not reliably create the group named `appuser` , and a subsequent `chown` lands on an invalid group. The LLM, given the error tail, replaces those two lines with:

```
RUN addgroup --system appuser \
    && adduser --system --no-create-home --ingroup appuser appuser \
    && chown -R appuser:appuser /app
```

and attempt 1 builds clean. This is the kind of fix a developer would Google for, framed as a 30-second LLM round trip.

Operating cost

Every run logs token usage and an estimated dollar cost at the model's paid-tier rate. A representative single-container run uses about 1,400 input tokens, 300 output tokens, and costs roughly \$0.0011 at paid-tier rates on Groq's Llama 3.3 70B. On Groq's free tier the cost is zero, with a daily ceiling well above what a single developer needs.

Provider	Free-tier ceiling	Default model
Groq	High daily request limit	<code>llama-3.3-70b-versatile</code>
Gemini	20 requests per day on Flash, none on Pro	<code>gemini-2.5-flash</code>

The LLM layer handles 429 responses by parsing the provider's retry hint and sleeping, applies a 60-second per-request timeout, and retries once on transient errors.

Engineering practices

Practice	Status
Test coverage	57 unit tests, including the source-code env-var grep, rate-limit logic, cost estimation, fence stripping, URL parsing, cache round-trip, and Pydantic schema validation
Continuous integration	GitHub Actions matrix on Python 3.10 through 3.13, plus Bandit security scan and Ruff lint, on every push and PR
Lint	Ruff with sensible defaults; zero warnings on <code>main</code>
Static security	Bandit with <code>-ll</code> ; zero high-confidence findings on <code>main</code>
Dependency hygiene	Dependabot watching pip and GitHub Actions weekly
Rate limiting	Three-layer in-process limiter on the public preview deployment (per-session cooldown, per-session cap, per-instance hourly cap), bypassed when a visitor brings their own API key
Input validation	Strict <code>github.com</code> URL check in both CLI and web UI before any network call
Disk hygiene	Old runs in <code>output/</code> are pruned on every new run, default keep 20
Secrets	Lives in <code>.env</code> (chmod 600, gitignored) locally, in Streamlit Cloud Secrets when deployed
License	MIT

Tech stack

Layer	Tool
Language	Python 3.10 or higher
LLM SDK	<code>google-genai</code> for Gemini, <code>groq</code> for Groq
Data contracts	Pydantic v2
CLI	Typer plus Rich
Web UI	Streamlit
Containerization	Docker Engine 20 plus, Docker Compose v2
Source control automation	<code>gh</code> CLI for fork and PR creation
Build / lint / test	pytest, pytest-cov, ruff, bandit

Features beyond the core pipeline

- **Bring your own key.** The deployed UI accepts a visitor's Gemini or Groq key in a sidebar field. With a key, rate limits are removed for that session.
- **autodock pr <run_dir> .** After a successful run, fork the upstream repo and open a pull request with the generated `Dockerfile` , `autodock.yaml` , and `docker-compose.yml` . Skips the fork when the upstream is owned by the same GitHub user.
- **autodock list** . Recent runs with attempt counts and outcomes, rendered as a Rich table in the terminal.
- **Sample-repo buttons in the web UI.** One-click loading of Flask, Node, and Go sample repos.
- **Profile cache.** Reruns on the same commit SHA skip the analyze LLM call.
- **Cost meter.** Token totals and an estimated dollar cost printed at the end of every run and persisted to `output/<run_id>/usage.json` .

Roadmap

Worth shipping next, ordered by impact:

1. Sandboxed builds so the public preview can run `docker build` safely on arbitrary repos, not only the dry-run preview that the current Streamlit Cloud container supports.
2. Real telemetry on the live deploy, opt-in, so it is clear which repos visitors actually try and what fails.
3. A short looping demo recording in the README. The recipe is in `docs/recording_demo.md` .
4. More multi-service compose evidence: a Flask plus Postgres run alongside the existing Flask plus Redis demo.
5. Anthropic Claude as a third LLM backend for users with that account.

Where to find things

Try it (preview mode)	https://auto-dock-it.streamlit.app
Code, issues, releases	https://github.com/MelvinJoshua1375/auto-dock-it
Demos with attempt history	https://github.com/MelvinJoshua1375/auto-dock-it/tree/main/demos
Reference PR from <code>autodock pr</code>	https://github.com/MelvinJoshua1375/autodock-pr-test/pull/1
Changelog	https://github.com/MelvinJoshua1375/auto-dock-it/blob/main/CHANGELOG.md
Contributing guide	https://github.com/MelvinJoshua1375/auto-dock-it/blob/main/CONTRIBUTING.md